

EE332 数字系统设计

Lab 3–3: 信号与变量的差异

姓 名: 何衍宁

学 号: 12311512

2026 年 3 月 12 日

目 录

1	实验目的	2
2	VHDL 代码和 testbench 代码	2
3	结果推导	3
4	结果	4
4.1	behavior simulation	4
4.2	post synthesis timing simulation	5
4.3	post implementation timing simulation	6
5	分析	6
6	结论	7

1 实验目的

本实验旨在通过 VHDL 建模与 Testbench 仿真，深入探究在单一进程（Process）中，变量的立即赋值机制（:=）与信号的调度赋值机制（<=）在处理中间逻辑时，对电路最终逻辑输出及仿真波形所产生的具体差异。

2 VHDL 代码和 testbench 代码

design 采用代码如下：

```
1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3
4  entity signal_vare is
5      Port ( x, y, z : in std_logic;
6            res1, res2 : out std_logic);
7  end signal_vare;
8
9  architecture dataflow of signal_vare is
10 signal sig_s1, sig_s2 : std_logic;
11 begin
12     proc1: process (x, y, z) is
13         variable var_s1, var_s2 : std_logic;
14     begin
15         L1: var_s1 := x and y;
16         L2: var_s2 := var_s1 xor z;
17         L3: res1 <= var_s1 nand var_s2;
18     end process proc1;
19
20     proc2: process (x, y, z) is
21     begin
22         L1: sig_s1 <= x and y;
23         L2: sig_s2 <= sig_s1 xor z;
24         L3: res2 <= sig_s1 nand sig_s2;
25     end process proc2;
26
27
28 end dataflow;
```

仿真 testbench 的代码如下：

```
1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3
4  entity testbench is
5  end testbench;
6
7  architecture tb of testbench is
8  component signal_vare is
9      Port ( x : in STD_LOGIC;
10            y : in STD_LOGIC;
11            z : in STD_LOGIC;
12            res1 : out STD_LOGIC;
13            res2 : out STD_LOGIC);
14  end component;
15  -- 初始化为 0
```

```

16 signal x, y, z: std_logic := '0' ;
17 signal r1, r2 : std_logic;
18 constant period: time := 10ns;
19
20 begin
21     uut: signal_vare port
22         map( x => x,
23             y => y,
24             z => z,
25             res1 => r1,
26             res2 => r2);
27     -- 周期 80ns
28     process
29     begin
30         wait for 40 ns;
31         x <= not x;
32     end process;
33     -- 周期 40ns
34     process
35     begin
36         wait for 20 ns;
37         y <= not y;
38     end process;
39     -- 周期 20ns
40     process
41     begin
42         wait for 10 ns;
43         z <= not z;
44     end process;
45
46 end tb;

```

3 结果推导

$$\text{res1} = \text{res2} = \overline{x \cdot y} + z \quad (1)$$

在 proc1 中，由于采用变量进行立即赋值，代码按顺序执行时会实时读取上一行的最新结果，从而构成了纯粹的组合逻辑。因此，输出 res1 仅在输入为 x=1、y=1 且 z=0 时才输出 0，其波形直观表现为在常态高电平中，每隔 80ns 出现一个宽度为 10ns 的低电平脉冲。

相比之下，proc2 采用的是信号与调度赋值 (<=)，其状态更新具有滞后性，在赋值瞬间信号值并不会立刻改变，进程内的后续逻辑仍沿用触发前的旧值参与计算。受这种非即时更新特性的影响，内部逻辑状态无法像变量那样同步传递，导致最终输出 res2 永远恒定为高电平 1。

4 结果

仿真得到的结果如下：

4.1 behavior simulation

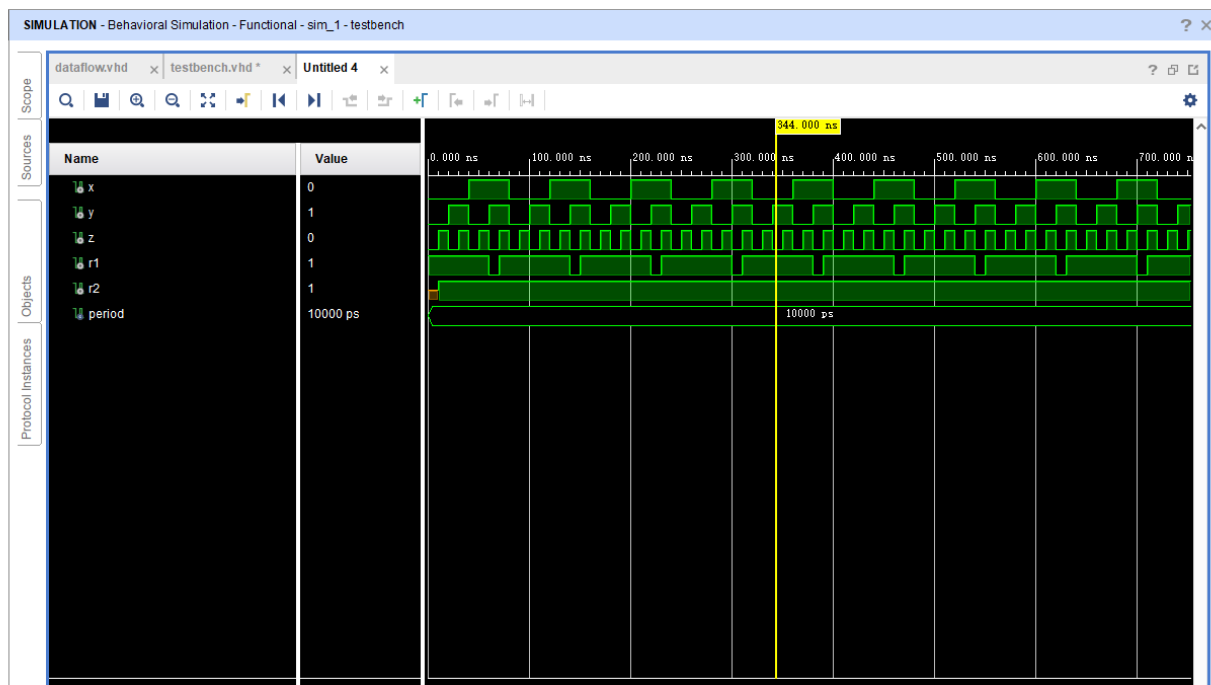


图 1: Behavior Simulation Result

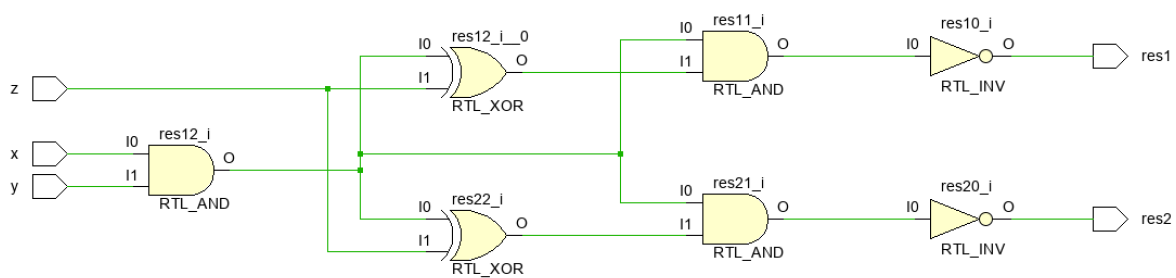


图 2: Behavior Schematic Simulation Result

4.2 post synthesis timing simulation

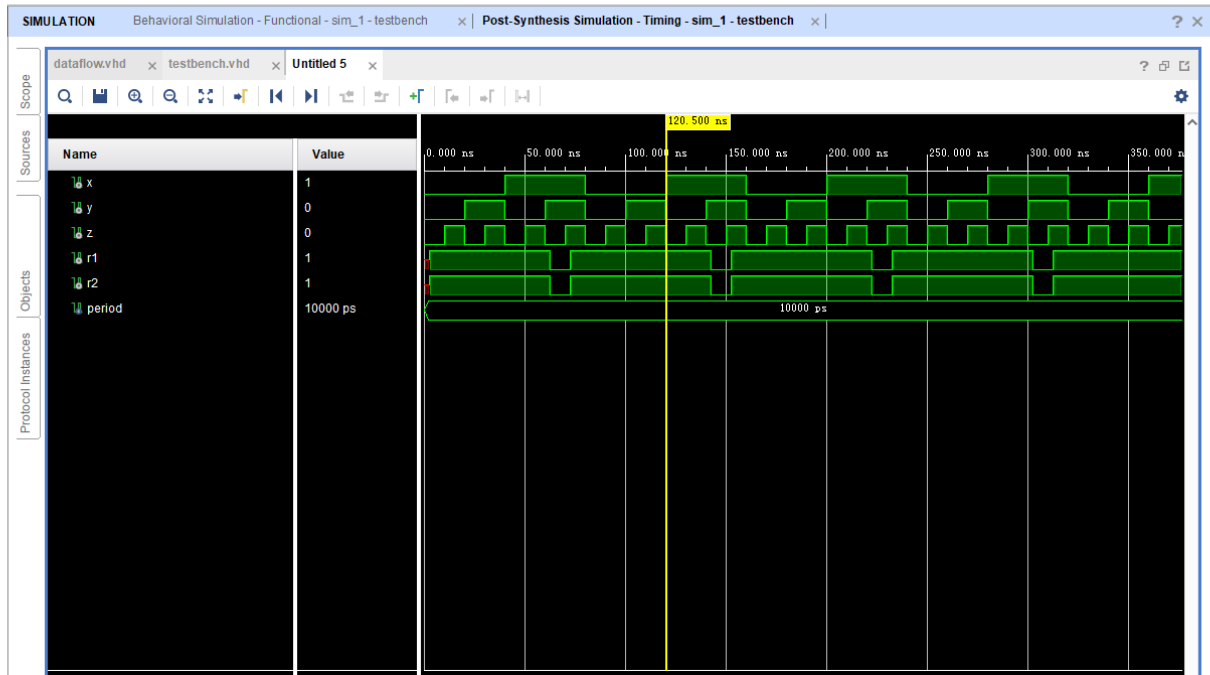


图 3: Post Synthesis Timing Simulation Result

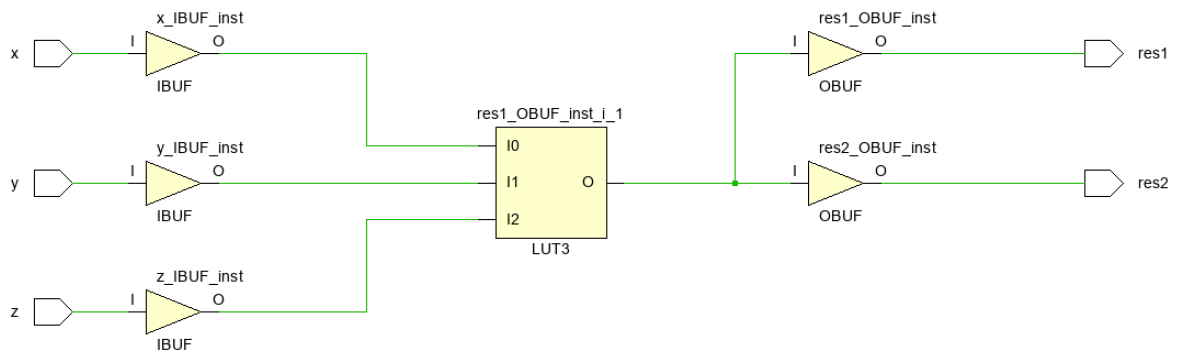


图 4: Synthesis Schematic Simulation Result

4.3 post implementation timing simulation

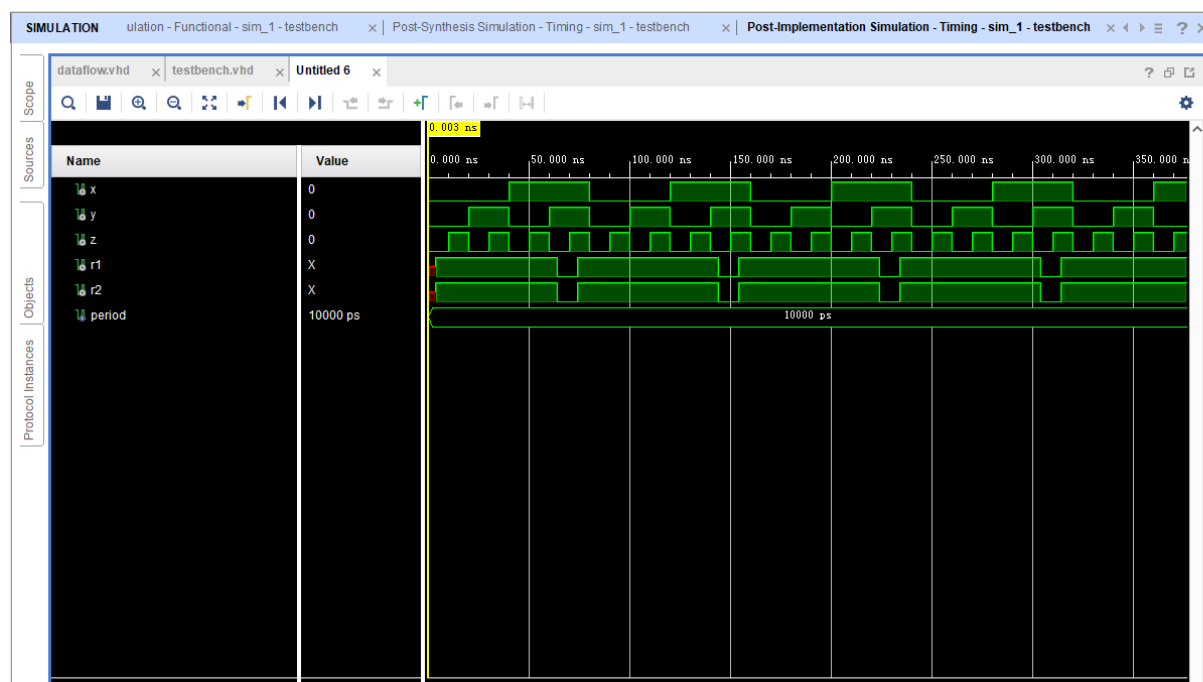


图 5: Post Implementation Timing Simulation Result

5 分析

本次实验的仿真波形深刻揭示了 VHDL 语言中变量与信号在底层调度机制上的本质差异：采用变量立即赋值（`:=`）的 `proc1` 能够完美复现预期的组合逻辑状态，其根本原因在于变量的更新在代码顺序执行时是瞬时生效的，这使得进程内的后续级联语句能够无缝、实时地调用前序运算的最新中间值。

与之形成强烈反差的是，基于信号调度赋值（`<=`）的 `proc2` 最终输出了完全违背预期的恒定高电平，这是因为信号的更新严格受制于“延迟分配”机制，即赋值动作仅被暂存于事件队列中，直到当前进程执行完毕并挂起（遇到 `end process`）时才会触发统一重载。这种非即时的滞后更新特性，直接导致代码在当前周期顺序向下执行时，内部信号 `sigs2` 与最终输出 `res2` 的计算被迫提取了上一个仿真周期（Delta Cycle）遗留的陈旧状态参与运算，从而引发了底层逻辑链路在时间轴上的严重错位，致使系统在任何单一时间切片内都无法凑齐输出低电平的先决条件，彻底破坏了原定的组合逻辑功能。

6 结论

本实验通过设计和仿真一个数字电路，成功展示了 VHDL 中 signal 与 variable 类型的区别。实验中正确实现了 VHDL 代码，并利用 testbench 验证了电路功能。理论分析与仿真结果一致，证明电路运行正确。实验表明，在数字系统设计中理解两者的不同行为非常重要：signal 适用于模块之间的通信和状态保持，而 variable 更适合在进程内部进行计算。